

University
Level

Object Oriented Programming

OOP Notes



- ★ Easy to Understand
- ★ Well Explained
- ★ Important Examples
- ★ University Syllabus Covered
- ★ Best for Exams



Learn → Understand → Code → Succeed

Handwritten Notes

♥ by ♥

@Hello_Universe



Introduction to OOP

→ What is OOP ?

OOP stands for **Object Oriented Programming**.

It is a programming paradigm based on the concept of "Objects", which can contain data (in the form of fields) and code (in the form of methods).

Example

A real world example of object is "Mobile". It has data like brand, color, price etc. and behaviors like calling, messaging, camera etc.

→ Who invented OOP ?

The concept of OOP was introduced by **Alan Kay** in 1960s. It was further developed by many programmers.

Alan Kay ★

→ Why OOP was created ?

- ★ To manage large and complex programs easily.
- ★ To make code reusable.
- ★ To improve security and data hiding.
- ★ To make software easy to maintain and modify.

Before OOP

Programming was done using functions. It was difficult to manage large projects. ☹️

→ Purpose of OOP

- ★ To organize code in a better way.
- ★ To solve real life problems using objects.
- ★ To connect program with real world.

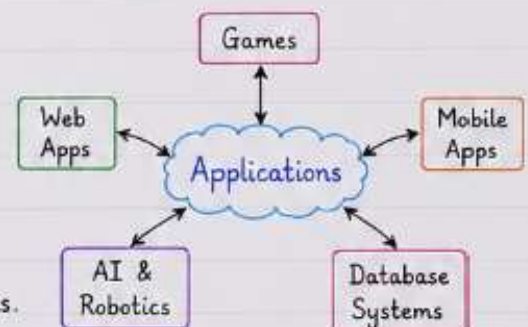
Real World

OOP

Solution

→ Where OOP is used ?

- ★ **Games** : 3D games, racing games etc.
- ★ **Web Development** : dynamic websites and applications.
- ★ **Mobile Apps** : Android apps, iOS apps.
- ★ **Database Systems** : real time data handling.
- ★ **AI & Robotics** : simulation and real world applications.



Features of OOP

- | | |
|---------------------|-------------------|
| ★ Class and Objects | ★ Inheritance |
| ★ Encapsulation | ★ Polymorphism |
| ★ Abstraction | ★ Dynamic Binding |

We will study each feature in detail in upcoming lectures.





Class : _____

OOP

Lecture : _____

Date : _____

2

Classes

A class is a **user-defined data type** that acts as a blueprint or template for creating objects. It defines the properties (data members) and behaviors (member functions) that the objects will have.

Note : Class is a logical entity.

Objects

An object is an **instance of a class**. It is a real-world entity that has state (values) and behavior (functions) as defined by the class.

Note : Object is a real-world entity.

> Difference between Class & Object <

Class	Object
1. Class is a blueprint or template.	1. Object is an instance of a class.
2. It is a logical entity.	2. It is a real-world entity.
3. No memory is allocated.	3. Memory is allocated.
4. It cannot perform any task.	4. It can perform tasks.
5. Declared using keyword "class".	5. Created from a class.
6. Example: <code>class Student {...};</code>	6. Example: <code>Student obj;</code>

Syntax

1. Class Syntax

```
class ClassName
{
    access_specifier:
    data_members;
    member_functions();
};
```

Example :

```
class Student
{
    public:
        string name;
        int rollNo;
        void show();
};
```

2. Object Syntax

```
ClassName objectName;
```

Example :

```
Student s1;
```

Real-life Examples

- Class : Car → Object : Toyota Car
- Class : Mobile → Object : iPhone 14
- Class : Student → Object : Ali
- Class : Book → Object : Physics Book
- Class : Employee → Object : John

Important : In OOP, we first design the class, then create objects from it and use them.

Access Specifiers ☆

Access specifiers are used to **control the visibility and accessibility** of class members (data members and member functions) from outside the class.

In C++, there are **three types** of access specifiers :

1. Public

2. Private

3. Protected

1. Public

- ☆ Members declared as public are accessible from **anywhere**.
- ☆ Can be accessed inside the class, outside the class, and by derived classes.
- ☆ It is the least restrictive access level.

Syntax:

```
class ClassName
{
    public:
        // members
};
```

Note : Public members form the **interface** of the class.

2. Private

- ☆ Members declared as private are accessible only **inside the class**.
- ☆ Cannot be accessed from outside the class or by derived classes directly.
- ☆ It is the most restrictive access level.

Syntax:

```
class ClassName
{
    private:
        // members
};
```

Note : Private members are **hidden** from the outside world.

3. Protected

- ☆ Members declared as protected are accessible **inside the class** and by **derived classes**.
- ☆ Cannot be accessed from outside the class.
- ☆ It provides a middle level of restriction.

Syntax:

```
class ClassName
{
    protected:
        // members
};
```

Note : Protected members are useful in **inheritance**.

Quick Comparison ☆

Access Specifier	Inside Class	Outside Class	Derived Class	Level
Public	✓	✓	✓	Least Restrictive
Private	✓	✗	✗	Most Restrictive
Protected	✓	✗	✓	Medium Restrictive

Always choose the access specifier carefully to ensure **data hiding** and **security**. ♥

Constructors ☆

A constructor is a special member function of a class that is **automatically called** when an object of the class is created.

It is used to **initialize** the data members of the class.

Note : Constructors have the same name as the class name and do not have any return type (not even void).

Types of Constructors ☆

C++ provides the following types of constructors :

1. Default Constructor

2. Parameterized Constructor

3. Copy Constructor

4. Dynamic Constructor

1. Default Constructor

- ☆ Called automatically when an object is created and no arguments are passed.
- ☆ It initializes the data members with default values.
- ☆ If we do not define any constructor, C++ provides a default constructor implicitly.

Syntax:

```
class ClassName
{
public:
    ClassName(); // default constructor
};
```

Example:

```
class Student
{
public:
    string name;
    int rollNo;

    Student() // default constructor
    {
        name = "Unknown";
        rollNo = 0;
    }
};
```

2. Parameterized Constructor

- ☆ Takes arguments and initializes the data members with the provided values.
- ☆ It allows us to set different values for different objects.
- ☆ Can be called multiple times with different arguments.

Syntax:

```
class ClassName
{
public:
    ClassName(type1 p1, type2 p2, ...); // parameterized constructor
};
```

Example:

```
class Student
{
public:
    string name;
    int rollNo;

    Student(string n, int r) // parameterized constructor
    {
        name = n;
        rollNo = r;
    }
};
```

Default vs Parameterized Constructor ☆

Feature	Default Constructor	Parameterized Constructor
Arguments	No arguments	Takes arguments
Purpose	Initializes with default values	Initializes with user-defined values
Called	Automatically (when no arguments passed)	Explicitly (with arguments)
Flexibility	Less flexible	More flexible
Example Use	Simple initialization	Custom initialization

Note : Constructors are executed once when the object is created.



Destructor ☆

- ☆ A destructor is a special member function of a class that is **automatically called** when an object is destroyed.
- ☆ It is used to **release memory**, close files, or perform any **cleanup activities**.
- ☆ Destructor has the same name as the class but is preceded by a **tilde (~)** symbol.

Key Point



Destructor does **not** take any arguments and does **not** return any value.

Syntax :

```
~ClassName()
{
    // cleanup code
}
```

Example :

```
class Student
{
public:
    ~Student()
    {
        cout << "Destructor called!\n";
    }
};
```

Destructor is called **automatically** when the object goes out of scope.

Constructor vs Destructor ☆

Feature	Constructor	Destructor
1. Purpose	Initializes the object and allocates resources.	Destroys the object and releases resources.
2. Name	Same as the class name.	Same as the class name but preceded by ~ (tilde).
3. Arguments	Can have arguments (default, parameterized etc.)	Does not take any arguments.
4. Return Type	Does not return any value.	Does not return any value.
5. When Called	Automatically called when object is created.	Automatically called when object is destroyed.
6. Frequency	Called once per object creation.	Called once per object destruction.
7. Use	For initialization of data members.	For cleanup activities like free memory, close files.

Constructor Example

```
#include <iostream>
using namespace std;
class Demo
{
public:
    int x;
    Demo()           // Constructor
    {
        x = 0;
        cout << "Constructor Called!\n";
    }
};
int main()
{
    Demo d;          // Constructor called
    return 0;
}
```

Remember



Constructor sets up the object.

Destructor cleans up the object.



Destructor Example

```
#include <iostream>
using namespace std;
class Demo
{
public:
    ~Demo()           // Destructor
    {
        cout << "Destructor Called!\n";
    }
};
int main()
{
    Demo d;           // d is created
                      // d is destroyed here
}

// Destructor called automatically
```

Note : Constructors are called **automatically** when objects are **created**, and Destructors are called **automatically** when objects are **destroyed**.



Encapsulation

- Encapsulation is one of the fundamental principles of **Object Oriented Programming**.
- It is the process of **wrapping (binding)** data members and member functions into a **single unit (class)**.
- It restricts direct access to some of an object's components and prevents accidental modification of data.
- Encapsulation improves **data security, integrity** and code maintainability.

How Encapsulation Achieves Data Hiding ?

By declaring the data members as **private** and providing **public getter** and **setter** methods, we can control access to the data.

- **Private** data members cannot be accessed directly from outside the class.
- **Public** methods act as an interface to access or modify the data.

Data Hiding

- Data hiding is the technique of hiding the implementation details (data) from the outside world.
- Only **essential information** is provided to the user, while the internal details are **hidden**.
- It is achieved through access specifiers (**private, protected**).

Without Data Hiding (Not Encapsulated)

```
class Student
{
public:
    int rollNo;
    string name;
    int marks;
};
```

Problem:

Anyone can directly access and modify the data.

With Data Hiding (Encapsulated)

```
class Student
{
private:
    int rollNo;
    string name;
    int marks;
public:
    void setRollNo(int r) { rollNo = r; }
    int getRollNo() { return rollNo; }
    void setName(string n) { name = n; }
    string getName() { return name; }
    void setMarks(int m) { marks = m; }
    int getMarks() { return marks; }
};
```

Advantage:

Data is accessed and modified only through public methods (controlled access).

Encapsulation vs Data Hiding

Feature	Encapsulation	Data Hiding
Definition	Binding of data members and member functions into a single unit.	Hiding the implementation details (data) from the outside world.
Focus	Combines data and functions together.	Restricts access to data.
Achieved By	Using classes and access specifiers.	Using private/protected members and public methods.
Purpose	To achieve security, reusability and easy maintenance.	To prevent unauthorized access and accidental modification.
Relation	Encapsulation includes data hiding.	Data hiding is part of encapsulation.

Real-life Example

Example: Mobile Phone

We call someone using a mobile phone.

We only use buttons (interface) to call, message, or take photos.

We don't know how the internal system (processor, memory, code) works.

This is **Encapsulation** and **Data Hiding** in real life.

Abstraction

→ What is Abstraction?

Abstraction is one of the fundamental concepts of OOP.

It means showing only the **essential features** (what an object does) and hiding the **internal implementation details** (how it does).

It helps in **reducing complexity** and **increasing efficiency**.

Key Points

- Focuses on **what** an object does, not **how** it does.
- Hides the unnecessary details from the user.
- Achieved using **abstract classes** and **interfaces**.
- Helps in **reducing complexity** and **improving maintainability**.

→ Explanation

In abstraction, we define a set of methods that should be performed by any sub-class, but we **do not specify how** these methods are implemented.

The implementation is left to the sub-classes.

Example : Consider a vehicle.

We know that every vehicle has some common behavior like **start()**, **stop()**, **accelerate()** etc. But the way these actions are performed may differ from vehicle to vehicle (car, bike, bus).

So, we **show only the common features** and **hide the internal working**.

Advantages of Abstraction

- ① **Reduces Complexity** : It hides unnecessary details and shows only important features.
- ② **Improves Security** : Hides the internal implementation, so data remains safe from unwanted access.
- ③ **Easy Maintenance** : Changes in internal implementation do not affect the external code.
- ④ **Reusability** : Abstraction allows reusing code at different places.
- ⑤ **Better Design** : It helps in designing a clear and simple system.

In Short

Abstraction improves program structure, hides complexity, and helps developers to focus on **what** an object should do instead of **how** it does.

Inheritance

→ What is Inheritance?

Inheritance is one of the important features of OOP.

It allows a new class (**derived class**) to acquire the properties and behaviors (methods) of an existing class (**base class**).

It promotes **code reusability** and establishes a relationship between classes.

Why Inheritance is Used?

- ① It supports **code reusability**.
- ② It improves the **readability** and **maintainability** of code.
- ③ It allows us to **add more features** to an existing class without modifying it.
- ④ It represents **real-life relationship** (is-a relationship).

Syntax

```
class DerivedClass : access_specifier BaseClass
{
    // members of derived class
};
```

access_specifier can be:

- **public** : public members of base class remain **public** in derived class.
- **protected** : public and protected members of base class remain **protected** in derived class.
- **private** : public and protected members of base class become private in derived class.

Example

```
class Animal
{
public:
void eat()
{
    cout << "Animal is eating.\n";
}
};
```

```
class Dog : public Animal
{
public:
void bark()
{
    cout << "Dog is barking.\n";
}
};
```

Explanation:

Dog is the **derived class** (**child class**) and **Animal** is the **base class** (**parent class**). **Dog** inherits **eat()** method from **Animal** and also has its own method **bark()**.

Important Points

- Inheritance establishes an **is-a relationship**.
- A derived class can access **public** and **protected** members of the base class.
- Private members of the base class **cannot be accessed** directly in the derived class.
- The derived class can **add its own members** and functions.
- Inheritance is **transitive**, if A is parent of B and B is parent of C, then A is also parent of C.

Note : Inheritance should be used **carefully**.

Overuse of inheritance can make the program **complex**.

Types of Inheritance

Inheritance has several types based on the relationship between **base class** (parent) and **derived class** (child).

①. Single Inheritance

A derived class is created from a single base class.

In this type, one child class inherits properties and methods from one parent class.

Example : class B is derived from class A.

②. Multiple Inheritance

A derived class is created from more than one base class.

In this type, one child class inherits properties and methods from two or more parent classes.

Example : class C is derived from class A and class B.

③. Multilevel Inheritance

A derived class is created from another derived class.

In this type, a child class is derived from a class that is already derived from another class.

Example : class C is derived from class B, and class B is derived from class A.

④. Hierarchical Inheritance

More than one derived class is created from a single base class.

In this type, multiple child classes inherit from the same parent class.

Example : class B and class C are derived from class A.

⑤. Hybrid Inheritance

A combination of two or more types of inheritance.

In this type, a combination of different inheritance types is used in a program.

Example : Combination of Multiple and Multilevel inheritance.

Important Points

- Inheritance promotes **code reusability** and reduces redundancy.
- It helps in creating a **clear relationship** between classes.
- It supports the concept of **real-life relationships**.
- Overuse of inheritance can make the program **complex**.

Note : Inheritance represents "is-a" relationship.
It is one of the key features of **Object Oriented Programming**.

Polymorphism

→ What is Polymorphism?

Polymorphism is one of the most important features of OOP.

The word "Polymorphism" means "many forms".

It allows objects of different classes to be treated as objects of a common base class.

It allows a **single interface** to be used for a general class of actions.

Key Points

- It increases **flexibility** and **extensibility** of the program.
- It allows methods to do **different things** based on the object that it is acting upon.
- It can be achieved in **two ways** : Compile time polymorphism and Runtime polymorphism.

→ Types of Polymorphism

Polymorphism is mainly of two types in C++ :

1. Compile time polymorphism (Static Binding)
2. Runtime polymorphism (Dynamic Binding)

① Compile Time Polymorphism (Static Binding)

In this type, the decision about which function to call is taken by the **compiler** at **compile time**.

It is also called **Static polymorphism**.

Achieved by :

1. Function Overloading
2. Operator Overloading

Example : (Function Overloading)

```
int add(int a, int b);  
int add(int a, int b, int c);
```

Both functions have same name "**add**" but different parameters.
Compiler decides which one to call.

② Runtime Polymorphism (Dynamic Binding)

In this type, the decision about which function to call is taken at **runtime**.

It is also called **Dynamic polymorphism**.

It is achieved by method **Overriding** (**Virtual Functions**).

Key Point :

- It is resolved at runtime.
- It is based on inheritance.
- It uses virtual functions.

Example : (Method Overriding)

A base class has a function "**display()**".
A derived class redefines (overrides) this function. The function to be called is decided at runtime.

Note : Polymorphism makes the program more **flexible** and **easier to extend**.
It allows us to write **one interface** and have **multiple implementations**.

Function Overloading

→ What is Function Overloading ?

Function Overloading is a feature of OOP in C++ where more than one function can have the **same name** but **different parameter list**.

These functions may have different number of parameters or different types of parameters or both.

Key Points

- Functions must have the **same name**.
- The **number of parameters** must be different, or
- The **type of parameters** must be different, or
- The **order of parameters** must be different.
- **Return type** can be same or different. It does not matter.

→ How it Works ?

When a function call is made, the compiler checks the arguments (number, types, and order) and decides which function to call.

This decision is taken at compile time. Therefore, function overloading is an example of **compile time polymorphism**.

Example

```
#include <iostream>
using namespace std;
int add(int a, int b)
{
    return a + b;
}
int add(int a, int b, int c)
{
    return a + b + c;
}
double add(double a, double b)
{
    return a + b;
}
int main()
{
    cout << add(10, 20) << endl;    // calls first add
    cout << add(10, 20, 30) << endl; // calls second add
    cout << add(5.5, 4.5) << endl;  // calls third add
    return 0;
}
```

Explanation :

Here, all three functions have the same name 'add' but different parameter lists. The compiler identifies which function to call based on the arguments passed.

Important Points

- Function overloading improves code **readability**.
- It allows us to use a **single name** for **similar operations**.
- It does not depend on **return type** only.
- It is achieved at **compile time**.

Note : Function Overloading is **not** the same as **Default Arguments**.

In overloading, compiler decides which function to call, whereas in default arguments, the same function is called with some parameters having default values.

Operator Overloading

→ What is Operator Overloading ?

Operator Overloading is a feature of OOP in C++ that allows us to use **existing operators** with **user-defined data types**.

It gives **extended meaning** to operators beyond their **built-in** operations.

We can redefine an operator so that it works with objects the way we want.

Key Points

- It improves **code readability**.
- It makes the program **easy to understand and maintain**.
- It provides a **natural way** to work with objects.
- Only **existing operators** can be overloaded. New operators **cannot** be created.
- At least one operand must be of **user-defined type**.

→ How it Works ?

When an operator is applied to objects, the compiler calls a special function (operator function) defined by the programmer.

This function defines how the operator works with objects.

Syntax

```
return_type operator symbol (parameter_list)
{
    // code
}
```

Example of Operators that can be Overloaded

+, -, *, /, %, ==, !=, <, >, <=, >=, =, ++, --, [], (), ->, <<, >> etc.

Example

```
#include <iostream>
using namespace std;

class Complex
{
    int real, imag;
public:
    Complex(int r = 0, int i = 0)
    {
        real = r;
        imag = i;
    }
    // Overloading + operator
    Complex operator + (Complex c2)
    {
        Complex temp;
        temp.real = real + c2.real;
        temp.imag = imag + c2.imag;
        return temp;
    }
    void display()
    {
        cout << real << " + " << imag << "i" << endl;
    }
};
```

Explanation :

Here, we overloaded the + operator. When we use + between two Complex objects, the operator +() function is called, which adds the real and imag parts and returns the result.

Important Points

- Operator overloading **does not change** the **original meaning** of an operator.
- It is achieved using **member function** or **friend function**.
- We cannot change the **precedence and associativity** of operators.
- Some operators **cannot** be overloaded like ::, ., .*, ?:
- It makes the program **more intuitive** and **user friendly**.

Note : Operator Overloading is **not** the same as overloading functions.

Function overloading is about multiple functions with the same name, while operator overloading is about giving **special meaning** to operators.

Virtual Functions and Friend Function

→ 1. Virtual Functions

Virtual functions are used to achieve **run-time polymorphism**. A member function is declared as **virtual** in the base class and redefined (overridden) in the derived class. The decision about which function to call is taken at **runtime**, not at compile time.

Example

```
class Base
{
public:
    virtual void show()
    { cout << "Base class\n"; }
};

class Derived : public Base
{
public:
    void show()
    { cout << "Derived class\n"; }
};
```

Syntax

```
class Base
{
public:
    virtual void display()
    {
        // code
    }
};
```

How it Works ?

When we call `show()` using a base class pointer referring to derived class object, the **derived class version of `show()`** is executed.

Important Points

- Virtual functions are declared using keyword **virtual**.
- They are used for **run-time polymorphism**.
- Base class pointer can point to **derived class object**.

→ 2. Friend Function

A friend function is a function that is not a member of the class but can access **private and protected** members of the class. It is declared using the keyword **friend** inside the class.

Example

```
class Box
{
private:
    int value;
public:
    Box(int v) { value = v; }
    friend void display(Box &b); // friend function
};

void display(Box &b)
{
    cout << "Value is: " << b.value << endl;
}
```

Syntax

```
class ClassName
{
private:
    int x;
public:
    friend return_type friendFunction(ClassName &obj);
};
```

Important Points

- Friend function is **not** a member of the class.
- It can access **private and protected** members.
- It is declared inside the class using **friend** keyword.
- It breaks **data hiding** concept, so it should be used carefully.
- It is not called using object or pointer.

Note : Virtual functions provide **run-time polymorphism**, while **friend** functions provide **special access** to class members.

This Pointer and Static Members

→ 1. This Pointer

The this pointer is a **special pointer** available in all non-static member functions. It is used to point to the **current object** whose function is being called.

Key Points

- this pointer **holds the address** of the object that invokes the member function.
- It is passed as a **hidden parameter** to the member function.
- It is used to **differentiate** between class members and local variables having the same name.
- It **returns** the current object.

Example

```
class Demo
{
private:
    int value;
public:
    Demo(int value)
    {
        this->value = value; // this value = local value
    }
    void display()
    {
        cout << "Value = " << this->value << endl;
    }
};
```

How it Works ?

When an object calls a member function, this pointer automatically points to that object.

If there is any ambiguity between object's data member and local variable, this pointer is used to refer to the **current object**.

Important Points

- this pointer is **not available** in **static** functions.
- It **cannot** be declared explicitly.
- It is **constant** (its value cannot be changed).

→ 2. Static Members

Static members are those members of a class that **belong to the class** rather than to any specific object. There is only **one copy** of static member shared by all objects.

Types of Static Members

1. Static Data Members (variables)
2. Static Member Functions

Key Points

- Static data members are **initialized only once**.
- They are **shared by all objects** of the class.
- Static member functions can access **only** static data members and **cannot access** non-static data members directly.
- Static members are declared using the keyword **static**.

Example

```
class Counter
{
private:
    static int count; // static data member
public:
    Counter()
    {
        count++; // increment for each object
    }
    static void displayCount() // static member function
    {
        cout << "Total Objects = " << count << endl;
    }
};

int Counter::count = 0; // initialize static member

int main()
{
    Counter c1, c2, c3; // three objects created
    Counter::displayCount(); // call using class name
    return 0;
}
```

Note : Static members are useful when we want to maintain data **common to all objects**, such as counting total objects, tracking shared information, etc.

File Handling in OOP (Read / Write Files)

→ What is File Handling ?

File handling in C++ allows a program to create, open, read, write and close files. In OOP, files are handled using classes from `<fstream>` library.

The `fstream` library provides three classes:

- `ifstream` → used to read data from files.
- `ofstream` → used to write data into files.
- `fstream` → used for both read and write.

1. Writing to a File

We use `ofstream` class to write data into a file.

Syntax :

```
ofstream outFile("filename.txt");
```

Example :

```
#include <fstream>
using namespace std;

int main()
{
    ofstream outFile("data.txt");
    outFile << "Hello World!" << endl;
    outFile << "This is file handling in OOP.";
    outFile.close();
    return 0;
}
```

Explanation :

This program creates a file named `data.txt` and writes two lines into it.

2. Reading from a File

We use `ifstream` class to read data from a file.

Syntax :

```
ifstream inFile("filename.txt");
```

Example :

```
#include <fstream>
#include <iostream>
using namespace std;

int main()
{
    ifstream inFile("data.txt");
    string line;
    while (getline(inFile, line))
    {
        cout << line << endl;
    }
    inFile.close();
    return 0;
}
```

Explanation :

This program opens `data.txt` and reads it line by line using `getline()`.

Common File Modes

Mode	Description
<code>ios::in</code>	Open file for reading
<code>ios::out</code>	Open file for writing (Creates new file)
<code>ios::app</code>	Open file for appending (Add data at end)
<code>ios::binary</code>	Open file in binary mode
<code>ios::ate</code>	Open file and move pointer to end

Important Points

- Always close the file after operation using `close()`.
- If file does not exist, it will be created in writing mode.
- Use `getline()` to read full line including spaces.
- File handling helps in data persistence.

Note : File handling is very useful in real-world applications for storing and retrieving data permanently.

Exception Handling

→ What is Exception Handling ?

Exception handling in C++ is a mechanism to handle **runtime errors** (exceptions) in a program so that the program does not crash unexpectedly.

It separates error-handling code from normal code.

It helps in **maintaining the flow** of program.

Key Points

- Exceptions occur at **runtime**.
- They disrupt the normal flow of the program.
- Exception handling handles these errors gracefully.
- It increases the **reliability** of program.

→ Basic Syntax

```
try
{
    // code that may throw exception
}
catch (exceptionType e)
{
    // code to handle exception
}
```

How it Works ?

The code inside **try** block is executed.

If an error (exception) occurs, the normal flow stops and the control is transferred to the **catch** block.

The **catch** block handles the exception.

After handling, the program continues normally.

1. try

- The **try** block contains the code that may generate an exception.
- If no exception occurs, **catch** block is skipped.
- If exception occurs, **catch** block is executed.

2. catch

- The **catch** block catches and handles the exception.
- It contains the code to deal with the error.
- Multiple **catch** blocks can be used for different exceptions.

3. throw

- The **throw** keyword is used to throw an exception manually.
- It transfers control to the nearest **catch** block.
- Any data (value/message) can be thrown.

Example Program

```
#include <iostream>
using namespace std;
int main()
{
    int a, b, result;
    cout << "Enter numerator : ";
    cin >> a;
    cout << "Enter denominator : ";
    cin >> b;

    try
    {
        if(b == 0)
            throw "Denominator cannot be zero";
        result = a / b;
        cout << "Result = " << result << endl;
    }
    catch(const char* msg)
    {
        cout << "Error : " << msg << endl;
    }
    return 0;
}
```

Explanation

- Code inside **try** block is executed.
- If user enters 0 as denominator, an exception is thrown using **throw**.
- This exception is caught by **catch** block and error message is displayed.
- Program does not crash and continues smoothly.

Common Example of Exceptions

- Division by zero
- Invalid input
- Null pointer access
- File not found
- Array index out of bounds

Note : Exception handling makes the program more **robust** and **user-friendly**. It prevents abnormal termination of program.

Inline Functions

→ What is Inline Function ?

An inline function is a function in which the compiler **replaces** the function call with the **actual code** of the function to execute.

It **avoids the overhead** of function call (time and stack memory).

It is used for **small and frequently called** functions.

Key Points

- Improves program speed.
- Reduces function call overhead.
- Best for small functions.
- Code size may increase.

→ Syntax :

```
inline return_type function_name(parameters)
{
    // code
}
```

Important Note

Using inline is just a **request** to the compiler, **not a command**. Compiler decides whether to inline or not.

→ How it Works ?

When a program calls an inline function, the compiler inserts (copies) the body of the function at the place of the call. So no jump to function, no return back.

This **saves time and memory**.

→ Example Program

```
#include <iostream>
using namespace std;
// inline function
inline int add(int a, int b)
{
    return a + b;
}
int main()
{
    int x = 5, y = 10, sum;
    sum = add(x, y);    // function call
    cout << "Sum = " << sum << endl;
    return 0;
}
```

Explanation

- The function **add()** is declared as **inline**.
- When **add(x, y)** is called, the compiler **replaces** the call with code: **return x + y;**
- This **avoids function call overhead**.

Example Without Inline

If we do **not** use **inline**, then:

- Control goes to function.
- Code executes.
- Control returns back.
(This takes time and memory.)

→ When to Use Inline Functions ?

- When function is **small** (1-5 lines).
- When function is called **many times**.
- When **performance** is important.
- **Not suitable for large functions** (code size increases).

Advantages

- Increases execution speed.
- Saves time of function call.
- Useful for small, simple functions.

Note : Overuse of inline may **increase the size** of the program and may reduce overall performance.

≡ Most Repeated Question ≡

- ① What is **Object Oriented Programming**? Explain its main features.
- ② Differentiate between **Class** and **Object** with examples.
- ③ What is a **Constructor**? Explain Default and Parameterized Constructor.
- ④ Explain **Inheritance** and its types in C++.
- ⑤ What is **Polymorphism**? Explain Compile Time and Runtime Polymorphism.
- ⑥ Explain **Encapsulation** and **Data Hiding** with example.
- ⑦ What is **Abstraction**? Write its advantages.
- ⑧ Explain **Function Overloading** with syntax and example.
- ⑨ What is **Virtual Function** in C++? Why is it used?
- ⑩ Explain **Exception Handling** using **try**, **catch** and **throw**.

♡ THE END ♡

•—————•
@ Hello_Universe ♡